

Building a FuSa Documentation Automation Machine

Luke Renchik
Independent

21 April 2026

Abstract

Functional Safety evaluation and documentation are problems of data synthesis and data reasoning, which are prone to human error and require large ramp-up efforts. The goal of this tool is to synthesize, chunk, and categorically store all relevant information related to a product and leverage the use of LLM reasoning systems to assist in the evaluation and creation of the necessary documentation for these safety critical systems.



Contents

1	Part I — Foundations	2
1.1	Designing Data-Intensive Systems	2
1.1.1	Networking Protocols	2
1.1.2	Data-Intensive Application Fundamentals	4
2	Part I — Foundations	10
2.1	Designing Data-Intensive Systems	10
2.1.1	Networking Protocols	10
2.1.2	Data-Intensive Application Fundamentals	12

1 Part I — Foundations

Prerequisites and background knowledge required before implementation.

1.1 Designing Data-Intensive Systems

Started: 4/19/26

1.1.1 Networking Protocols

TCP (Transmission Control Protocol).

TCP operates at the transport layer. Its job is to take raw IP packets — which are inherently unreliable and unordered — and deliver a reliable, ordered byte stream between two endpoints.

Connection Lifecycle

1. Three-Way Handshake — establishes a connection before any data flows:

```
Client → SYN      (I want to connect, my seq# is X)
Server → SYN-ACK   (OK, my seq# is Y, I acknowledge X)
Client → ACK       (Acknowledged Y)
```

2. Data Transfer

- TCP numbers every byte with a sequence number
- Receiver sends ACKs confirming what was received
- Sender retransmits anything not ACKed within a timeout
- Reorders out-of-order segments using sequence numbers before handing to the application

TCP throttles transmission rate via:

- **Receive window** — how much the receiver can buffer
- **Congestion window** — how much the network can handle

3. Teardown — uses a four-way FIN/ACK exchange. Either side can initiate.

—

HTTP (Hypertext Transfer Protocol).

HTTP is the application protocol on top of TCP. It doesn't know or care about packets — it just writes bytes to a TCP socket and reads bytes back.

Request / Response Format

```
GET /api/users HTTP/1.1
Host: example.com
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 83
```

```
{"users": [...]}
```

HTTP Versions

Speed	Slower due to overhead	Faster, minimal overhead
Use Cases	HTTP, SSH, email, file transfer	Video streaming, gaming, DNS, VoIP, QUIC

UDP is essentially raw IP with port numbers — speed and control, but you implement any reliability you need yourself (which is exactly what QUIC does).

—

QUIC.

Transport protocol built on UDP, designed by Google and standardized in RFC 9000. HTTP/3 is HTTP/2's semantics running over QUIC instead of TCP.

- **Stream isolation** — packet loss only affects the one stream it belonged to
- **Connection IDs** — instead of IP + port tuples, so connections survive switching from WiFi to LTE
- **TLS 1.3 mandatory and baked in** — handshake is combined with the transport handshake, reducing round trips
- **0-RTT resumption** — for returning connections, data can flow before the handshake even completes

—

TLS 1.3.

Encryption layer beneath HTTPS. Version 1.3 (2018) was a significant cleanup over 1.2.

- **Faster handshake** — TLS 1.2 required 2 round trips before data could flow. TLS 1.3 does it in 1 round trip. With 0-RTT resumption, a returning client can send data with the very first packet.
- **Removed legacy cruft** — TLS 1.2 supported dozens of cipher suites including broken ones (RC4, MD5, SHA-1). TLS 1.3 cuts to 5 modern suites only.
- **Forward secrecy is mandatory** — if your private key is compromised in the future, recorded past sessions still can't be decrypted, because each session used a fresh ephemeral key.

—

1.1.2 Data-Intensive Application Fundamentals

DDIA — Chapter 1

The three pillars of any well-designed data-intensive system:

Pillar	Concern
Reliability	Tolerating hardware & software faults, human error
Scalability	Measuring load & performance, latency, percentiles, throughput
Maintainability	Operability, simplicity, & evolvability

Core functions of data-intensive applications:

Store Data	Databases
Speed Up Reads	Caches — remember the result of an expensive operation
Search & Filter	Search indexes — keyword search and filtering
Batch Processing	Periodically crunch large amounts of accumulated data

—

Redis (Remote Dictionary Server).

An open source, in-memory data structure store usable as a database, cache, message broker, and streaming engine. Keeps data primarily in RAM — typically handles millions of operations per second.

Common Use Cases

- **Caching layer** — sits in front of a slow database so frequent queries skip the DB entirely
- **Rate limiting** — track requests per IP using atomic increment operations
- **Pub/Sub** — lightweight async communication between services
- **Distributed locks** — SET NX allows services to coordinate shared resource access without race conditions

Race Conditions

A race condition occurs when program correctness depends on the relative timing of concurrent operations. Two or more threads “race” to read and modify shared state, with outcomes differing depending on who wins.

Thread A reads balance: \$150
Thread B reads balance: \$150 ← B reads before A has written
Thread A writes balance: \$50
Thread B writes balance: \$50 ← B overwrites A's result

Both withdrawals succeed, but the final balance is \$50 instead of \$-50. The window of vulnerability between a read and a subsequent write is called a **critical section**. Race conditions are non-deterministic — they may only surface under load, on specific hardware, or once in a million requests.

Atomic Operations

An operation is **atomic** if it executes as a single, indivisible unit — no observable intermediate state exists.

Thread A: reads \$150, writes \$50 (atomic)
Thread B: reads \$50 → rejects withdrawal (insufficient funds)

How atomicity is achieved:

- **Hardware (CAS)** — Compare-And-Swap: “only write if the current value is what I expect.” The foundation everything else builds on.

- **Mutexes/locks** — only one thread enters the critical section at a time
- **Semaphores** — generalized locks allowing N concurrent accessors
- **Atomic types** — e.g., `std::atomic<int>` in C++; hardware-backed, no explicit locking needed
- **Database ACID** — atomicity is the “A”: either all statements in a transaction commit or none do
- **Redis** — every command is atomic; Redis processes commands sequentially in a single thread

Wherever you have shared state and concurrency, reason carefully about what operations need to be atomic and over what scope.

—

Apache Kafka.

Core Abstraction: The Log

Kafka models message streams as an **append-only log**, exactly like a database’s write-ahead log (WAL).

No deletion on read	Messages persist for a configurable retention window
Offsets	Every message has a monotonically increasing integer index
Independent consumers	Each consumer tracks its own offset; can rewind and replay
Sequential writes	Append-only writes are extremely fast on disk

Terminology

- **Topic** — named stream of messages, analogous to a database table
- **Partition** — each topic split into N independent logs for parallelism

Topic: "orders"

```
Partition 0: [msg0] [msg3] [msg6] ...
Partition 1: [msg1] [msg4] [msg7] ...
Partition 2: [msg2] [msg5] [msg8] ...
```

- **Broker** — a single Kafka server; clusters run 3–N brokers for fault tolerance
- **Replication** — each partition has a leader + N replicas; writes go to leader, replicated before ACK

—

Application-Managed Caching & Memcached.

The application is explicitly responsible for what to cache, when to invalidate, and how to handle misses. The cache itself is a dumb key-value store with no knowledge of business logic.

Cache-Aside Pattern (Lazy Loading)

```
def get_user(user_id):
    cached = cache.get(f"user:{user_id}")
    if cached:
        return cached                                # cache hit
```

```

user = db.query("SELECT * FROM users WHERE id = ?", user_id)
cache.set(f"user:{user_id}", user, ttl=300)    # store for 5 min
return user                                    # cache miss, now populated

```

Cache Invalidation — one of the hardest problems in CS. Cache goes stale when:

- Underlying data changes and the cache is not updated
- Multiple services write to the same data through different paths
- A distributed system has inconsistent views across nodes

Strategies: **TTL expiration, explicit invalidation on write, versioned keys.**

Memcached — oldest and most widely deployed caching system. Focused entirely on speed and simplicity.

- Stores only strings (client serializes everything)
- No persistence, no replication, no server-side clustering logic
- Pure LRU eviction when memory is full
- **Distributed hashing** — client library maps keys to nodes via consistent hashing; deterministic and coordination-free
- **Slab allocation** — memory pre-divided into fixed-size chunk classes to avoid heap fragmentation

Memcached vs. Redis

Choose when...	Memcached	Redis
Data structures	Plain strings only	Lists, sets, hashes, streams, ...
Persistence	Not needed	Required (RDB / AOF)
Clustering	Client-managed simplicity	Server-managed built-in
Threading	True multi-threaded	Mostly single-threaded
Extra features	Not needed	Pub/Sub, scripting, transactions

Full Stack View

```

User Request
  ↓
Application Server
  ↓
Memcached / Redis ← cache-aside for DB query results, sessions
  ↓ (on miss)
PostgreSQL / MySQL ← source of truth
  ↓ (on writes)
Kafka ← emit change events for downstream consumers

```

—

Full-Text Search.

Elasticsearch or **Solr** — allow users to search data by keyword or filter in various ways. *(To be expanded)*

Building a FuSa Documentation Automation Machine

Luke Renchik
Independent

21 April 2026

Abstract

Functional Safety evaluation and documentation are problems of data synthesis and data reasoning, which are prone to human error and require large ramp-up efforts. The goal of this tool is to synthesize, chunk, and categorically store all relevant information related to a product and leverage the use of LLM reasoning systems to assist in the evaluation and creation of the necessary documentation for these safety critical systems.



Contents

1	Part I — Foundations	2
1.1	Designing Data-Intensive Systems	2
1.1.1	Networking Protocols	2
1.1.2	Data-Intensive Application Fundamentals	4
2	Part I — Foundations	10
2.1	Designing Data-Intensive Systems	10
2.1.1	Networking Protocols	10
2.1.2	Data-Intensive Application Fundamentals	12

2 Part I — Foundations

Prerequisites and background knowledge required before implementation.

2.1 Designing Data-Intensive Systems

Started: 4/19/26

2.1.1 Networking Protocols

TCP (Transmission Control Protocol).

TCP operates at the transport layer. Its job is to take raw IP packets — which are inherently unreliable and unordered — and deliver a reliable, ordered byte stream between two endpoints.

Connection Lifecycle

1. Three-Way Handshake — establishes a connection before any data flows:

```
Client → SYN      (I want to connect, my seq# is X)
Server → SYN-ACK   (OK, my seq# is Y, I acknowledge X)
Client → ACK       (Acknowledged Y)
```

2. Data Transfer

- TCP numbers every byte with a sequence number
- Receiver sends ACKs confirming what was received
- Sender retransmits anything not ACKed within a timeout
- Reorders out-of-order segments using sequence numbers before handing to the application

TCP throttles transmission rate via:

- **Receive window** — how much the receiver can buffer
- **Congestion window** — how much the network can handle

3. Teardown — uses a four-way FIN/ACK exchange. Either side can initiate.

—

HTTP (Hypertext Transfer Protocol).

HTTP is the application protocol on top of TCP. It doesn't know or care about packets — it just writes bytes to a TCP socket and reads bytes back.

Request / Response Format

```
GET /api/users HTTP/1.1
Host: example.com
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 83
```

```
{"users": [...]}
```

HTTP Versions

Speed	Slower due to overhead	Faster, minimal overhead
Use Cases	HTTP, SSH, email, file transfer	Video streaming, gaming, DNS, VoIP, QUIC

UDP is essentially raw IP with port numbers — speed and control, but you implement any reliability you need yourself (which is exactly what QUIC does).

—

QUIC.

Transport protocol built on UDP, designed by Google and standardized in RFC 9000. HTTP/3 is HTTP/2's semantics running over QUIC instead of TCP.

- **Stream isolation** — packet loss only affects the one stream it belonged to
- **Connection IDs** — instead of IP + port tuples, so connections survive switching from WiFi to LTE
- **TLS 1.3 mandatory and baked in** — handshake is combined with the transport handshake, reducing round trips
- **0-RTT resumption** — for returning connections, data can flow before the handshake even completes

—

TLS 1.3.

Encryption layer beneath HTTPS. Version 1.3 (2018) was a significant cleanup over 1.2.

- **Faster handshake** — TLS 1.2 required 2 round trips before data could flow. TLS 1.3 does it in 1 round trip. With 0-RTT resumption, a returning client can send data with the very first packet.
- **Removed legacy cruft** — TLS 1.2 supported dozens of cipher suites including broken ones (RC4, MD5, SHA-1). TLS 1.3 cuts to 5 modern suites only.
- **Forward secrecy is mandatory** — if your private key is compromised in the future, recorded past sessions still can't be decrypted, because each session used a fresh ephemeral key.

—

2.1.2 Data-Intensive Application Fundamentals

DDIA — Chapter 1

The three pillars of any well-designed data-intensive system:

Pillar	Concern
Reliability	Tolerating hardware & software faults, human error
Scalability	Measuring load & performance, latency, percentiles, throughput
Maintainability	Operability, simplicity, & evolvability

Core functions of data-intensive applications:

Store Data	Databases
Speed Up Reads	Caches — remember the result of an expensive operation
Search & Filter	Search indexes — keyword search and filtering
Batch Processing	Periodically crunch large amounts of accumulated data

—

Redis (Remote Dictionary Server).

An open source, in-memory data structure store usable as a database, cache, message broker, and streaming engine. Keeps data primarily in RAM — typically handles millions of operations per second.

Common Use Cases

- **Caching layer** — sits in front of a slow database so frequent queries skip the DB entirely
- **Rate limiting** — track requests per IP using atomic increment operations
- **Pub/Sub** — lightweight async communication between services
- **Distributed locks** — SET NX allows services to coordinate shared resource access without race conditions

Race Conditions

A race condition occurs when program correctness depends on the relative timing of concurrent operations. Two or more threads “race” to read and modify shared state, with outcomes differing depending on who wins.

Thread A reads balance: \$150
Thread B reads balance: \$150 ← B reads before A has written
Thread A writes balance: \$50
Thread B writes balance: \$50 ← B overwrites A's result

Both withdrawals succeed, but the final balance is \$50 instead of \$-50. The window of vulnerability between a read and a subsequent write is called a **critical section**. Race conditions are non-deterministic — they may only surface under load, on specific hardware, or once in a million requests.

Atomic Operations

An operation is **atomic** if it executes as a single, indivisible unit — no observable intermediate state exists.

Thread A: reads \$150, writes \$50 (atomic)
Thread B: reads \$50 → rejects withdrawal (insufficient funds)

How atomicity is achieved:

- **Hardware (CAS)** — Compare-And-Swap: “only write if the current value is what I expect.” The foundation everything else builds on.

- **Mutexes/locks** — only one thread enters the critical section at a time
- **Semaphores** — generalized locks allowing N concurrent accessors
- **Atomic types** — e.g., `std::atomic<int>` in C++; hardware-backed, no explicit locking needed
- **Database ACID** — atomicity is the “A”: either all statements in a transaction commit or none do
- **Redis** — every command is atomic; Redis processes commands sequentially in a single thread

Wherever you have shared state and concurrency, reason carefully about what operations need to be atomic and over what scope.

—

Apache Kafka.

Core Abstraction: The Log

Kafka models message streams as an **append-only log**, exactly like a database’s write-ahead log (WAL).

No deletion on read	Messages persist for a configurable retention window
Offsets	Every message has a monotonically increasing integer index
Independent consumers	Each consumer tracks its own offset; can rewind and replay
Sequential writes	Append-only writes are extremely fast on disk

Terminology

- **Topic** — named stream of messages, analogous to a database table
- **Partition** — each topic split into N independent logs for parallelism

Topic: "orders"

```
Partition 0: [msg0] [msg3] [msg6] ...
Partition 1: [msg1] [msg4] [msg7] ...
Partition 2: [msg2] [msg5] [msg8] ...
```

- **Broker** — a single Kafka server; clusters run 3–N brokers for fault tolerance
- **Replication** — each partition has a leader + N replicas; writes go to leader, replicated before ACK

—

Application-Managed Caching & Memcached.

The application is explicitly responsible for what to cache, when to invalidate, and how to handle misses. The cache itself is a dumb key-value store with no knowledge of business logic.

Cache-Aside Pattern (Lazy Loading)

```
def get_user(user_id):
    cached = cache.get(f"user:{user_id}")
    if cached:
        return cached                                # cache hit
```

```

user = db.query("SELECT * FROM users WHERE id = ?", user_id)
cache.set(f"user:{user_id}", user, ttl=300)    # store for 5 min
return user                                    # cache miss, now populated

```

Cache Invalidation — one of the hardest problems in CS. Cache goes stale when:

- Underlying data changes and the cache is not updated
- Multiple services write to the same data through different paths
- A distributed system has inconsistent views across nodes

Strategies: **TTL expiration, explicit invalidation on write, versioned keys.**

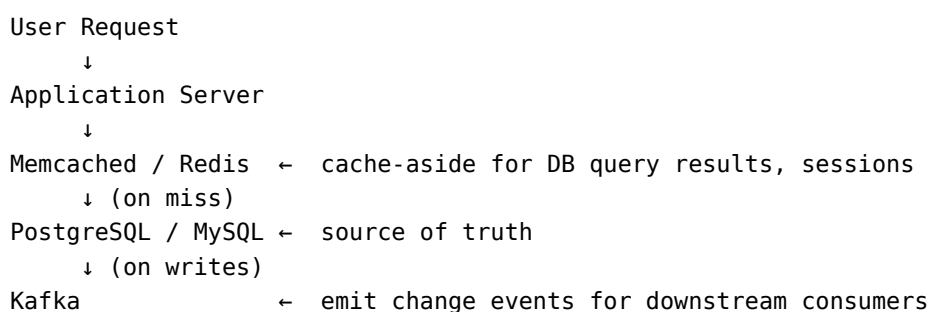
Memcached — oldest and most widely deployed caching system. Focused entirely on speed and simplicity.

- Stores only strings (client serializes everything)
- No persistence, no replication, no server-side clustering logic
- Pure LRU eviction when memory is full
- **Distributed hashing** — client library maps keys to nodes via consistent hashing; deterministic and coordination-free
- **Slab allocation** — memory pre-divided into fixed-size chunk classes to avoid heap fragmentation

Memcached vs. Redis

Choose when...	Memcached	Redis
Data structures	Plain strings only	Lists, sets, hashes, streams, ...
Persistence	Not needed	Required (RDB / AOF)
Clustering	Client-managed simplicity	Server-managed built-in
Threading	True multi-threaded	Mostly single-threaded
Extra features	Not needed	Pub/Sub, scripting, transactions

Full Stack View



—

Full-Text Search.

Elasticsearch or **Solr** — allow users to search data by keyword or filter in various ways. (*To be expanded*)